

Exp No:13 Experiment to handle missing values and inappropriate data in a
given data set

Aim:

To demonstrate the handling of missing and inappropriate data in a dataset using Python Pandas for data preprocessing.

Program Description:

This program creates a dataset using Pandas and performs data preprocessing operations such as identifying duplicate records, removing duplicates, resetting the index, removing unnecessary columns, identifying missing values, finding unique values, and replacing missing numerical values using suitable statistical methods such as mean and median.

Program:

```
import numpy as np

import pandas as pd

# Create the dataset

data = {

    "CustomerID": [1,2,3,4,5,6,7,8,9,10],

    "Age_Group": ["20-25","25-30","30-35","20-25","35+",

                  "35+","20-25","25-30","30-35","30-35"],

    "Rating(1-5)": [4,5,3,4,np.nan,5,4,2,5,4],

    "Hotel": ["Ibis","LemonTree","RedFox","LemonTree","Ibis",

              "Ibis","RedFox","LemonTree","Ibis","RedFox"],

    "FoodPreference": ["Veg","Non-Veg","Veg","Veg","Non-Veg",

                       "Veg","Non-Veg","Veg","Non-Veg","Non-Veg"],

    "Bill": [1500,2100,1350,np.nan,950,1800,1100,2400,3200,1750],

    "NoOfPax": [2,3,2,2,2,3,np.nan,3,4,2],

    "EstimatedSalary": [42000,56000,32000,60000,48000,

                        np.nan,35000,68000,91000,52000]

}
```



```

df = pd.DataFrame(data)

# Display dataset
print(df)

# The duplicated() method returns a Series with True and False
# values that describe which rows in the DataFrame are duplicated
df.duplicated()

print("\nDuplicate Values:")

print(df.duplicated())

# The info() method prints information about the DataFrame
df.info()

# Remove duplicate rows
df.drop_duplicates(inplace=True)

print("\nAfter Removing Duplicates:")

print(df)

# While removing duplicate record row index also removed
# The len() function returns the number of rows
print("\nLength of DataFrame:")

print(len(df))

# Reset the index
index = np.array(list(range(0, len(df))))
df.set_index(index, inplace=True)

print("\nNew Index:")

print(index)

print("\nDataFrame after Resetting Index:")

print(df)


# Axis refers to dimensions of DataFrame
# Drop unnecessary column if available

```



```
# Here we create an extra column for demonstration

df["Age_Group.1"] = df["Age_Group"]

print("\nBefore Dropping Age_Group.1:")
print(df)

df.drop(["Age_Group.1"], axis=1, inplace=True)

print("\nAfter Dropping Age_Group.1:")
print(df)

# loc is used for label indexing and accessing multiple columns

print("\nMissing CustomerID:")
print(df.loc[df["CustomerID"].isna()])

print("\nMissing Bill:")
print(df.loc[df["Bill"].isna()])

print("\nMissing EstimatedSalary:")
print(df.loc[df["EstimatedSalary"].isna()])

# Find missing values

print("\nMissing Values Count:")
print(df.isna().sum())

# Display unique values
```



```
print("\nUnique Age Groups:")  
print(df["Age_Group"].unique())
```

```
print("\nUnique Hotels:")  
print(df["Hotel"].unique())
```

```
# Display unique Food Preference  
print("\nUnique Food Preferences:")  
print(df["FoodPreference"].unique())
```

```
# Replace inappropriate value if present  
df["FoodPreference"].replace(  
    ["Vegetarian", "Veg"], ["Veg", "Veg"], inplace=True  
)
```

```
print("\nFood Preference after Replacement:")  
print(df["FoodPreference"])
```

```
# Fill missing values
```

```
# Numerical continuous data - use mean  
df["EstimatedSalary"].fillna(  
    round(df["EstimatedSalary"].mean()), inplace=True  
)
```

```
# Numerical discrete data - use median  
df["NoOfPax"].fillna(  
    df["NoOfPax"].median(), inplace=True
```



```
)
```

```
# Numerical data - use median
```

```
df["Rating(1-5)"].fillna(  
    round(df["Rating(1-5)"].median()), inplace=True  
)
```

```
# Numerical continuous data - use mean
```

```
df["Bill"].fillna(  
    round(df["Bill"].mean()), inplace=True  
)
```

```
# Final cleaned dataset
```

```
print("\nFinal Cleaned Dataset:")
```

```
print(df)
```



	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	\
0	1	20-25	4.0	Ibis	Veg	1500.0	
1	2	25-30	5.0	LemonTree	Non-Veg	2100.0	
2	3	30-35	3.0	RedFox	Veg	1350.0	
3	4	20-25	4.0	LemonTree	Veg	NaN	
4	5	35+	NaN	Ibis	Non-Veg	950.0	
5	6	35+	5.0	Ibis	Veg	1800.0	
6	7	20-25	4.0	RedFox	Non-Veg	1100.0	
7	8	25-30	2.0	LemonTree	Veg	2400.0	
8	9	30-35	5.0	Ibis	Non-Veg	3200.0	
9	10	30-35	4.0	RedFox	Non-Veg	1750.0	

	NoOfPax	EstimatedSalary
0	2.0	42000.0
1	3.0	56000.0
2	2.0	32000.0
3	2.0	60000.0
4	2.0	48000.0
5	3.0	NaN
6	NaN	35000.0
7	3.0	68000.0
8	4.0	91000.0
9	2.0	52000.0

Duplicate Values:

0	False
1	False
2	False
3	False
4	False
5	False
6	False
7	False
8	False
9	False

dtype: bool

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 10 entries, 0 to 9

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	CustomerID	10 non-null	int64
1	Age_Group	10 non-null	object
2	Rating(1-5)	9 non-null	float64
3	Hotel	10 non-null	object
4	FoodPreference	10 non-null	object
5	Bill	9 non-null	float64
6	NoOfPax	9 non-null	float64
7	EstimatedSalary	9 non-null	float64

dtypes: float64(4), int64(1), object(3)

memory usage: 772.0+ bytes

After Removing Duplicates:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	\
0	1	20-25	4.0	Ibis	Veg	1500.0	
1	2	25-30	5.0	LemonTree	Non-Veg	2100.0	
2	3	30-35	3.0	RedFox	Veg	1350.0	
3	4	20-25	4.0	LemonTree	Veg	NaN	



```

4      5      35+      NaN      Ibis      Non-Veg      950.0
5      6      35+      5.0      Ibis      Veg      1800.0
6      7      20-25     4.0      RedFox     Non-Veg     1100.0
7      8      25-30     2.0      LemonTree   Veg      2400.0
8      9      30-35     5.0      Ibis      Non-Veg     3200.0
9     10      30-35     4.0      RedFox     Non-Veg     1750.0

   NoOfPax  EstimatedSalary
0         2.0         42000.0
1         3.0         56000.0
2         2.0         32000.0
3         2.0         60000.0
4         2.0         48000.0
5         3.0           NaN
6        NaN         35000.0
7         3.0         68000.0
8         4.0         91000.0
9         2.0         52000.0

Length of DataFrame:
10

New Index:
[0 1 2 3 4 5 6 7 8 9]

DataFrame after Resetting Index:
   CustomerID  Age_Group  Rating(1-5)  Hotel  FoodPreference  Bill \

```

```

0         1      20-25     4.0      Ibis      Veg      1500.0
1         2      25-30     5.0      LemonTree   Non-Veg     2100.0
2         3      30-35     3.0      RedFox      Veg      1350.0
3         4      20-25     4.0      LemonTree   Veg       NaN
4         5      35+      NaN      Ibis      Non-Veg     950.0
5         6      35+      5.0      Ibis      Veg      1800.0
6         7      20-25     4.0      RedFox     Non-Veg     1100.0
7         8      25-30     2.0      LemonTree   Veg      2400.0
8         9      30-35     5.0      Ibis      Non-Veg     3200.0
9        10      30-35     4.0      RedFox     Non-Veg     1750.0

   NoOfPax  EstimatedSalary
0         2.0         42000.0
1         3.0         56000.0
2         2.0         32000.0
3         2.0         60000.0
4         2.0         48000.0
5         3.0           NaN
6        NaN         35000.0
7         3.0         68000.0
8         4.0         91000.0
9         2.0         52000.0

Before Dropping Age_Group.1:
   CustomerID  Age_Group  Rating(1-5)  Hotel  FoodPreference  Bill \
0         1      20-25     4.0      Ibis      Veg      1500.0
1         2      25-30     5.0      LemonTree   Non-Veg     2100.0

```



2	3	30-35	3.0	RedFox	Veg	1350.0
3	4	20-25	4.0	LemonTree	Veg	NaN
4	5	35+	NaN	Ibis	Non-Veg	950.0
5	6	35+	5.0	Ibis	Veg	1800.0
6	7	20-25	4.0	RedFox	Non-Veg	1100.0
7	8	25-30	2.0	LemonTree	Veg	2400.0
8	9	30-35	5.0	Ibis	Non-Veg	3200.0
9	10	30-35	4.0	RedFox	Non-Veg	1750.0

	NoOfPax	EstimatedSalary	Age_Group.1
0	2.0	42000.0	20-25
1	3.0	56000.0	25-30
2	2.0	32000.0	30-35
3	2.0	60000.0	20-25
4	2.0	48000.0	35+
5	3.0	NaN	35+
6	NaN	35000.0	20-25
7	3.0	68000.0	25-30
8	4.0	91000.0	30-35
9	2.0	52000.0	30-35

After Dropping Age_Group.1:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	\
0	1	20-25	4.0	Ibis	Veg	1500.0	
1	2	25-30	5.0	LemonTree	Non-Veg	2100.0	

2	3	30-35	3.0	RedFox	Veg	1350.0
3	4	20-25	4.0	LemonTree	Veg	NaN
4	5	35+	NaN	Ibis	Non-Veg	950.0
5	6	35+	5.0	Ibis	Veg	1800.0
6	7	20-25	4.0	RedFox	Non-Veg	1100.0
7	8	25-30	2.0	LemonTree	Veg	2400.0
8	9	30-35	5.0	Ibis	Non-Veg	3200.0
9	10	30-35	4.0	RedFox	Non-Veg	1750.0

	NoOfPax	EstimatedSalary
0	2.0	42000.0
1	3.0	56000.0
2	2.0	32000.0
3	2.0	60000.0
4	2.0	48000.0
5	3.0	NaN
6	NaN	35000.0
7	3.0	68000.0
8	4.0	91000.0
9	2.0	52000.0

Missing CustomerID:

Empty DataFrame

Columns: [CustomerID, Age_Group, Rating(1-5), Hotel, FoodPreference, Bill, NoOfPax, EstimatedSalary]

Index: []

Missing Bill:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	\
--	------------	-----------	-------------	-------	----------------	------	---------	---



```
3      4      20-25      4.0 LemonTree      Veg      NaN      2.0
```

```
EstimatedSalary
```

```
3      60000.0
```

```
Missing EstimatedSalary:
```

```
CustomerID Age_Group Rating(1-5) Hotel FoodPreference Bill NoOfPax \
5      6      35+      5.0 Ibis      Veg      1800.0      3.0
```

```
EstimatedSalary
```

```
5      NaN
```

```
Missing Values Count:
```

```
CustomerID      0
```

```
Age_Group      0
```

```
Rating(1-5)      1
```

```
Hotel      0
```

```
FoodPreference      0
```

```
Bill      1
```

```
NoOfPax      1
```

```
EstimatedSalary      1
```

```
dtype: int64
```

```
Unique Age Groups:
```

```
['20-25' '25-30' '30-35' '35+']
```

```
Unique Food Preferences:
```

```
['Veg' 'Non-Veg']
```

```
Food Preference after Replacement:
```

```
0      Veg
```

```
1      Non-Veg
```

```
2      Veg
```

```
3      Veg
```

```
4      Non-Veg
```

```
5      Veg
```

```
6      Non-Veg
```

```
7      Veg
```

```
8      Non-Veg
```

```
9      Non-Veg
```

```
Name: FoodPreference, dtype: object
```

```
Final Cleaned Dataset:
```

```
CustomerID Age_Group Rating(1-5) Hotel FoodPreference Bill \
0      1      20-25      4.0 Ibis      Veg      1500.0
1      2      25-30      5.0 LemonTree      Non-Veg      2100.0
2      3      30-35      3.0 RedFox      Veg      1350.0
3      4      20-25      4.0 LemonTree      Veg      1794.0
4      5      35+      4.0 Ibis      Non-Veg      950.0
5      6      35+      5.0 Ibis      Veg      1800.0
6      7      20-25      4.0 RedFox      Non-Veg      1100.0
```



7	8	25-30	2.0	LemonTree	Veg	2400.0
8	9	30-35	5.0	Ibis	Non-Veg	3200.0
9	10	30-35	4.0	RedFox	Non-Veg	1750.0

	NoOfPax	EstimatedSalary
0	2.0	42000.0
1	3.0	56000.0
2	2.0	32000.0
3	2.0	60000.0
4	2.0	48000.0
5	3.0	53778.0
6	2.0	35000.0
7	3.0	68000.0
8	4.0	91000.0
9	2.0	52000.0

```

/tmp/ipykernel_2070/3845947477.py:98: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or 'df[col] = df[col].method(value)' instead, to perform the operation inplace on the original object.

df["FoodPreference"].replace(
/tmp/ipykernel_2070/3845947477.py:108: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or 'df[col] = df[col].method(value)' instead, to perform the operation inplace on the original object.

df["EstimatedSalary"].fillna(
/tmp/ipykernel_2070/3845947477.py:113: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or 'df[col] = df[col].method(value)' instead, to perform the operation inplace on the original object.

df["NoOfPax"].fillna(
/tmp/ipykernel_2070/3845947477.py:118: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or 'df[col] = df[col].method(value)' instead, to perform the operation inplace on the original object.

df["Rating(1-5)"].fillna(
/tmp/ipykernel_2070/3845947477.py:123: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or 'df[col] = df[col].method(value)' instead, to perform the operation inplace on the original object.

df["Bill"].fillna(

```

Result:

Thus, the missing, duplicate, and inappropriate data in the dataset were successfully identified and handled using Python Pandas. The missing numerical values were replaced using suitable mean and median values, and the final cleaned dataset was obtained successfully.